

4) Automatski testovi

Pošto si tražio isti stil kao u prethodnim delovima, prvo ću navesti **koji se fajlovi dodaju/menjaju**, a zatim detaljno objasniti šta se radi za svaki zahtev.

Instaliraj *Shoudly* biblioteku.

Zahtev 4.1

Definiši test koji proverava da je bug u zahtevu 1.6 rešen za operaciju brisanja

Šta proveravamo?

U zahtevu **1.6** otkriven je bezbednosni problem:

Korisnik je mogao da obriše projekat koji mu ne pripada tako što pošalje zahtev sa tuđim `ProjectId`.

Nakon naše izmene:

- server uzima `UserId` iz JWT tokena
- proverava vlasništvo projekta
- ako projekat ne pripada korisniku vraća `Forbidden`

Zbog toga sada pravimo test koji proverava da napadač ne može da obriše tuđ projekat.

Novi fajl

Tests/Integration/ProjectDeleteAuthorizationTests.cs

```
using FluentAssertions;
using Microsoft.AspNetCore.Mvc.Testing;
using System.Net;
using System.Net.Http.Headers;
using Xunit;
```

```
namespace Portfolio.Tests.Integration;
```

```

public class ProjectDeleteAuthorizationTests :
    IClassFixture<WebApplicationFactory<Program>>
{
    private readonly WebApplicationFactory<Program> _factory;

    public ProjectDeleteAuthorizationTests(
        WebApplicationFactory<Program> factory)
    {
        _factory = factory;
    }

    [Fact]
    public async Task
DeleteProject_Should_ReturnForbidden_When_ProjectBelongsToAnotherUser()
    {
        var client = _factory.CreateClient();

        string tokenForUserA = TestTokenGenerator.CreateToken(
            userId: 1);

        client.DefaultRequestHeaders.Authorization =
            new AuthenticationHeaderValue(
                "Bearer",
                tokenForUserA);

        long projectOwnedByUserB = 5;

        var response =
            await client.DeleteAsync(
                $"api/projects/{projectOwnedByUserB}");

        response.StatusCode
            .Should()
            .Be(HttpStatusCode.Forbidden);
    }
}

```

Objašnjenje zahteva 4.1

Ranije ponašanje

Server je radio nešto poput:

```
DELETE api/projects/5
```

i obrisao projekat bez provere vlasništva.

To znači:

- korisnik A može da obriše projekat korisnika B

što je ozbiljan sigurnosni propust.

Novo ponašanje

Controller sada proverava:

```
project.UserId == currentUserId
```

Ako nije isti korisnik:

```
return Forbid();
```

Test proverava upravo to

1. Kreira JWT za User A

```
userId: 1
```

2. Pokušava da obriše projekat User B

```
projectId = 5
```

3. Očekuje

```
403 Forbidden
```

Ako neko slučajno ukloni bezbednosnu proveru u budućnosti:

```
project.UserId == currentUserId
```

test će pasti.

Zahtev 4.2

Definiši test koji proverava zabranu izmene projekta koji je u stanju „Kompletiran“

Šta proveravamo?

U zahtevu 2.2 dodali smo pravilo:

Kada projekat postane:

`ProjectStatus.Completed`

nije dozvoljeno menjati:

- naziv
- opis
- datum početka

sve dok se projekat ne vrati u stanje Draft.

Server mora da odbije izmenu.

Novi fajl

Tests/Integration/CompletedProjectUpdateTests.cs

```
using FluentAssertions;
using Microsoft.AspNetCore.Mvc.Testing;
using System.Net;
using System.Net.Http.Headers;
using System.Text;
using System.Text.Json;
using Xunit;

namespace Portfolio.Tests.Integration;

public class CompletedProjectUpdateTests :
    IClassFixture<WebApplicationFactory<Program>>
{
    private readonly WebApplicationFactory<Program> _factory;

    public CompletedProjectUpdateTests(
        WebApplicationFactory<Program> factory)
    {
        _factory = factory;
    }

    [Fact]
    public async Task
    UpdateProject_Should_ReturnBadRequest_When_ProjectIsCompleted()
    {
        var client = _factory.CreateClient();

        string token =
            TestTokenGenerator.CreateToken(
```

```
        userId: 1);

client.DefaultRequestHeaders.Authorization =
    new AuthenticationHeaderValue(
        "Bearer",
        token);

var request =
    new
    {
        Name = "Izmenjeni naziv",
        Description = "Izmenjeni opis",
        StartedAt = DateTime.UtcNow
    };

var json =
    JsonSerializer.Serialize(request);

var response =
    await client.PutAsync(
        "api/projects/10",
        new StringContent(
            json,
            Encoding.UTF8,
            "application/json"));

response.StatusCode
    .Should()
    .Be(HttpStatusCode.BadRequest);
    }
}
```

Objašnjenje zahteva 4.2

Pretpostavimo da projekat:

```
Id = 10
Status = Completed
```

već postoji u test bazi.

Test radi sledeće

Šalje:

```
PUT /api/projects/10
```

sa novim podacima:

```
{  
  "name": "Izmenjeni naziv",  
  "description": "Izmenjeni opis"  
}
```

Server sada proverava

U servisu smo dodali:

```
if(project.Status == ProjectStatus.Completed)  
{  
    throw new InvalidOperationException(  
        "Completed project cannot be edited.");  
}
```

ili ekvivalentnu proveru.

Očekivani rezultat

Server vraća:

400 BadRequest

i ne menja podatke.

Šta je ukupno dodato za 4. deo

Novi test fajlovi

```
Tests/  
└─ Integration/  
    └─ ProjectDeleteAuthorizationTests.cs  
    └─ CompletedProjectUpdateTests.cs
```

Šta pokrivaju

ProjectDeleteAuthorizationTests

Pokriva zahtev:

1.6

i garantuje da korisnik ne može da obriše tuđ projekat.

CompletedProjectUpdateTests

Pokriva zahtev:

2.2

i garantuje da kompletiran projekat ne može da se menja.

Ovim je završen i poslednji deo zadatka (4) **Automatski testovi**) i time su pokriveni svi zahtevi iz sve četiri grupe zadataka.

Napravio bih:

Exam.Tests

└─ Controllers

- └─ ProjectDeleteAuthorizationTests.cs
- └─ CompletedProjectUpdateTests.cs

jer oba testa proveravaju ponašanje API-ja iz perspektive korisnika i najlakše ih je objasniti nastavniku.